Защита проекта Aéza Услуги и продления

Текст для выступающего. Основан на критериях исходного кейса Invoice Keeper, страницы 10-17, и реализованной версии проекта. Рекомендованная длительность основного рассказа - примерно 6-8 минут, живой демонстрации - 3-4 минуты. Это предложение для репетиции, а не регламент организаторов: в исходном PDF лимит времени не указан. Обязательные семь действий из страницы 15 показаны в исходном порядке. Проект является решением кейса, адаптированным под Aéza; он не выдаётся за официальный подключённый сервис провайдера.

Основной рассказ

Слайд 1. Задача

Мы представляем «Aéza · Услуги и продления» - модуль учёта регулярных расходов с напоминаниями в Telegram. Он помогает команде видеть, что нужно оплатить, к какому сроку, что уже оплачено и где появилась просрочка.

Мы адаптировали исходный кейс под инфраструктурные услуги Aéza: серверы, хостинг и другие регулярные расходы. При этом сохранили обязательные правила задания: две роли, ежемесячные периоды, три валюты, история оплат и напоминания за три дня.

Слайд 2. Проблема и польза

Когда услуг несколько, сроки приходится держать в голове, переписке или таблице. Участники команды могут по-разному понимать, оплатили ли конкретный месяц. А отметка об одной оплате иногда заменяет всю историю.

Наше решение собирает сроки и отметки в одном месте. Администратор задаёт регулярный расход, система создаёт отдельные месячные периоды и напоминает о ближайшем сроке. Пользователь подтверждает оплату конкретного периода. Так команда получает общий ответ на три вопроса: что предстоит оплатить, что уже закрыто и что требует внимания.

Слайд 3. Граница с кабинетом Aéza

Сейчас это работающий модуль ручного учёта в оформлении Aéza. Названия, суммы и сроки вводятся в него вручную. Демонстрационные суммы условны и не являются тарифами провайдера.

Услугу оплачивают в официальном кабинете Aéza, после чего фиксируют факт в нашем модуле. Отметка «Оплачено» не списывает деньги и не продлевает сервер. Прекращение учёта не отключает услугу. Автоматический импорт счетов, баланса и SSO пока не подключены. Это позволяет честно отделить готовую часть проекта от следующего этапа интеграции.

Слайд 4. Роли и работа команды

У нас две роли. Администратор добавляет регулярные расходы, меняет будущие условия и управляет обычными пользователями. Его собственная учётная запись задаётся при запуске и через панель не редактируется.

Обычный пользователь видит общий список и может подтвердить оплату, отменить отметку или исправить фактическую дату. Эти ограничения проверяет сервер. Если пользователь попробует вызвать административный API напрямую, запрос будет отклонён, даже если обойти интерфейс.

Слайд 5. Серия и отдельные периоды

Ключевое решение в модели данных - разделить регулярный расход и его месячные периоды. Например, сервер - это одна серия, а октябрьская и ноябрьская оплаты - две отдельные записи со своими сроками и статусами.

Оплата октября не делает ноябрь оплаченным. Просрочка октября не мешает появиться следующему месяцу. Каждый период хранит собственную сумму и валюту, поэтому история не переписывается при изменении будущих условий. Прекращение серии сохраняет прошлые записи, но останавливает создание новых периодов и напоминаний.

Слайд 6. Даты и деньги

Сроки рассчитываются по календарю в едином часовом поясе, заданном в настройках запуска. Если исходный срок - 31 января, следующий будет в последний день февраля, а затем снова 31 марта. Мы сохраняем исходное число месяца, чтобы короткий февраль не сдвинул все следующие сроки.

В день срока просрочки ещё нет. Она появляется со следующего дня. Для оплаченного периода расчёт зависит от фактической даты: её исправление пересчитывает просрочку, но не меняет исходный срок.

Деньги хранятся целыми минимальными единицами, например копейками. RUB, USD и EUR считаются отдельно. Мы не складываем рубли с долларами и не добавляем ненужную для кейса конвертацию.

Слайд 7. Напоминания

Напоминания отправляет отдельный фоновый процесс. Открытая страница для этого не нужна. Он проверяет сроки каждые 15 секунд. За три календарных дня до оплаты появляется сообщение с названием, суммой, валютой и датой. Если расход добавили позже этого момента, он попадёт в ближайшую проверку.

Для каждого периода в базе создаётся единственное задание доставки. Повторная проверка или перезапуск не повторяют сообщение с уже подтверждённой отправкой. Оплаченный период и прекращённая серия перед отправкой исключаются.

Есть важная граница: если Telegram принял сообщение, а сетевой ответ потерялся, сервер не может достоверно узнать результат. Мы сохраняем такую отправку как неопределённую и автоматически её не повторяем. Администратор видит состояние в журнале доставки и сверяет чат. Поэтому мы не обещаем абсолютную доставку ровно один раз при любом сетевом сбое.

Слайд 8. Архитектура и стек

Интерфейс написан на React и TypeScript, сборка выполняется Vite. Сервер - Python и FastAPI. Данные хранятся в SQLite. Отдельно работают процесс напоминаний и обработчик команд Telegram. Остановка командного бота не останавливает проверку сроков.

Для командного модуля на одном сервере SQLite упрощает запуск и резервное копирование. Нам не нужен отдельный сервер базы или брокер очередей. При переходе на несколько серверов потребуется PostgreSQL и другой механизм блокировки заданий. Сейчас приложение развёрнуто на тестовом сервере, доступно по HTTPS, а процессы запускаются службами системы.

Слайд 9. API и защита

API - это способ, которым страница обращается к серверу. Когда пользователь нажимает «Оплачено», интерфейс отправляет запрос, сервер проверяет сессию и права, изменяет нужный период и возвращает результат.

Маршруты имеют общий префикс `/api/v1`. Реализованы обязательные операции входа, регулярных расходов, отдельных оплат, валютных итогов и пользователей. В поставке есть OpenAPI-файл и рабочая Swagger-документация.

Пароли хранятся в виде хешей. Сессионный cookie защищён от чтения JavaScript, а изменяющие запросы требуют CSRF-токен и допустимый адрес источника. Токен Telegram находится на сервере и в браузер не передаётся. Наш API уже обслуживает приложение; для обычной работы пользователю не нужно вызывать его вручную.

Слайд 10. Дополнительные возможности

Поверх обязательной части мы добавили календарь, поиск и фильтры, экспорт CSV, журнал действий и диагностику напоминаний. Условия серии можно поменять со следующего месяца, сохранив текущий период и историю.

В Telegram доступны сводка, продления, ближайшие сроки и просроченные периоды с навигацией кнопками. Бот показывает данные только в настроенном чате. Оплата подтверждается в веб-панели.

Интерфейс адаптирован к телефону. Оформление, локальные шрифты и спокойные анимации подстроены под визуальный язык Aéza. Движение учитывает системную настройку уменьшения анимации. Эти детали делают повседневную работу удобнее, но не заменяют корректность расчётов и прав.

Слайд 11. Проверки и передача

Проверки покрывают права доступа, независимость периодов, конец месяца, пересчёт просрочки, валютные суммы и поведение напоминаний при повторных проходах и ошибках. На стенде отдельно проверены HTTPS, реальные сообщения Telegram и сохранение состояний после перезапуска. Результаты и команды зафиксированы в документации проверки.

Для передачи подготовлены исходный код, инструкция запуска, `.env.example` без секретов, условные демонстрационные данные и спецификация API. Дальше покажем обязательный сценарий вживую: от входа администратора до следующего независимого периода.

Слайд 12. Живая демонстрация

Переключиться на приложение. Выполнить семь действий из следующего раздела. Слайд со сценарием оставить как опору, а не замену живому показу.

Слайд 13. Результат и следующий этап

В результате есть работающий путь: администратор добавляет расход, система напоминает, пользователь фиксирует оплату, история сохраняется, следующий месяц остаётся самостоятельным.

Следующий этап для внедрения в продукт Aéza - согласовать источник счетов и модель доступа, подключить официальный API биллинга и сопоставить реальные услуги с нашими периодами. Это отдельная интеграционная работа. Готовая основа уже решает обязательную задачу кейса и показывает, как команда будет контролировать регулярные расходы.

Слайд 14. Справочник API

Приложение к презентации. В основной рассказ не входит. Открывать при вопросе об обязательном контракте: вход, сессия, серии, отдельные периоды, итоги по валютам, управление обычными пользователями, Swagger и OpenAPI. Не зачитывать весь список маршрутов.

Слайд 15. Открыть проект

Завершить показ слайдом с QR-кодами. Сказать: «Здесь можно открыть веб-модуль и Telegram-бота. Адрес стенда - https://138.124.14.93/, бот - @aezainkeep_bot. Для финансовых данных веб-модуля нужна учётная запись; просмотр в боте разрешён настроенному чату». Во время вопросов оставить этот слайд на экране. Справочник API на слайде 14 открывается при необходимости. Перед выступлением проверить доступность тестового сервера.

Семь шагов живой демонстрации

Именно эти семь действий перечислены на странице 15 исходного кейса. Дополнительные проверки не должны вытеснить их из выступления.

Шаг	Действие на экране	Что сказать	Время
1	Показать закрытый список до входа, затем войти администратором	«До входа финансовые данные недоступны. Сейчас я вхожу администратором модуля».	20 с
2	Создать «VPS · Демонстрация», 1500 RUB, срок сегодня или через 1-3 дня	«Это условный расход. Задаю сумму, валюту и ближайший срок. Повторение ежемесячное».	35 с
3	Открыть настроенный Telegram-чат и дождаться нового сообщения	«Напоминание содержит название, сумму, валюту и срок. Расход добавлен в трёхдневном окне, поэтому попал в ближайшую проверку».	25 с
4	Выйти, войти обычным пользователем, отметить созданный период оплаченным и указать фактическую дату	«Вижу общий список, но не управляю сериями и пользователями. Подтверждаю оплату только этого периода».	40 с
5	Найти заранее подготовленный неоплаченный период со сроком в прошлом	«Этот срок уже прошёл. Просрочка начинается со следующего дня после срока».	20 с
6	Отметить его датой после срока, показать дни просрочки, исправить фактическую дату на день срока	«Платёж был отмечен с опозданием. Исправляю фактическую дату: просрочка стала нулевой, а плановый срок остался прежним».	35 с
7	Открыть следующий месяц для той же серии, затем историю	«Следующий период имеет собственный статус. Предыдущая оплата сохранена и не закрыла будущий месяц».	25 с

Общее время - около 3 минут 20 секунд, с запасом на переключение вкладок до 4 минут. Скорость внешнего Telegram не гарантируется.

Подготовка демонстратора

До выхода на защиту

1. Открыть приложение, Telegram и защищённую Swagger-документацию в отдельных вкладках. Проверить адрес стенда, действительность сертификата и доступность сервера непосредственно перед выступлением. У выданного тестового VPS ограниченный срок действия, его нужно проверить или продлить заранее через владельца кабинета.
2. Подготовить обычного пользователя. Пароли вводить без вывода на слайды и не хранить в презентации. Учётные записи модуля и официального кабинета Aéza разные.
3. Проверить, что запущены API, worker и командный бот, в окружении указан правильный часовой пояс и разрешённый Telegram-чат. Отправить `/menu` текущему `@aezainkeep_bot`.
4. Подготовить одну отдельную демосерию с неоплаченным прошлым периодом. Записать её имя и срок в личные заметки. Для шага 6 фактическая дата после срока должна быть не позднее сегодняшнего дня. Лучше использовать срок хотя бы два дня назад.
5. Убедиться, что для этой серии есть следующий период и его можно открыть через выбор месяца. Проверить на репетиции точные названия кнопок текущего интерфейса.
6. Для шага 2 создать новый расход только во время демонстрации. Повторное использование уже уведомлённого периода не вызовет новое напоминание. Не меняйте системное время, outbox или отметки доставки ради показа.
7. Подготовить короткую запись или скриншоты успешного прогона как резерв при сетевом сбое. Они должны быть прямо названы записью. Страница 15 требует живой показ, поэтому резервные материалы не стоит представлять как выполнение этого требования вживую.
8. Убрать из общего экрана `.env`, SSH-терминал, менеджер паролей, личную переписку и исходные изображения с доступами. Демоданные на экране должны оставаться условными.

Если Telegram не ответил сразу

Сказать: «Проверка выполняется фоновым процессом, отправка зависит от доступности Telegram. Сейчас проверю состояние задания». Посмотреть диагностику доставки в роли администратора. Не нажимать повторную отправку вслепую, не менять состояние `unknown` и не выдавать старое сообщение за новое. Если внешняя сеть недоступна, честно отметить сбой и показать заранее записанный успешный прогон как резерв. После этого закончить остальные живые шаги.

Если осталось время после семи шагов

Можно показать календарь и раздельные валютные итоги, Swagger, запрет административного запроса обычному пользователю, историю после прекращения серии или смену будущей суммы со следующего месяца. Эти действия являются дополнительными. SMTP, произвольные webhook, чек, годовой бюджет и кнопка подтверждения оплаты прямо в Telegram не реализованы и не демонстрируются как готовые возможности.

Короткий рассказ на три минуты

Этот вариант заменяет основной рассказ при коротком регламенте. Он не включает семь шагов живой демонстрации, на которые нужно отдельное время. Если общий лимит защиты составляет три минуты, полный сценарий из задания и подробный рассказ в него не помещаются; приоритет и регламент нужно уточнить у организаторов заранее.

«Мы представляем “Aéza · Услуги и продления” - модуль, который собирает регулярные расходы, историю оплат и напоминания в одном месте. Он отвечает на три вопроса: что нужно оплатить, что уже оплачено и где есть просрочка.

В проекте две роли. Администратор добавляет расходы и управляет обычными пользователями. Пользователь видит общий список, отмечает оплату и исправляет фактическую дату. Права проверяются сервером, поэтому прямой вызов API не даёт пользователю административные возможности.

Мы разделили регулярный расход и его месячные периоды. Оплата октября не закрывает ноябрь, а просрочка не мешает появиться следующему месяцу. Каждый период хранит собственные условия и отметку. Для срока на 31-е используется последний день короткого месяца, затем исходное число восстанавливается. В день срока просрочки ещё нет. Исправление фактической даты пересчитывает её, не меняя плановую дату. RUB, USD и EUR учитываются отдельно, деньги хранятся целыми минимальными единицами.

Отдельный фоновый процесс напоминает за три дня до срока. Если расход добавили позже, он попадает в ближайшую проверку. В базе есть единственное задание на период, а подтверждённая отправка не повторяется после нового прохода или перезапуска. При неопределённом сетевом результате мы не отправляем сообщение повторно автоматически: администратор сверяет состояние с Telegram. Это честная граница защиты от дублей при работе с внешним сервисом.

Интерфейс написан на React и TypeScript, сервер - на Python и FastAPI, база - SQLite. Для небольшого модуля на одном сервере этого достаточно без отдельной базы и брокера очередей. Работают календарь, фильтры, CSV, журнал и команды Telegram. Есть OpenAPI и Swagger, исходный код и инструкция запуска. На тестовом сервере проверены HTTPS, реальные уведомления и сохранение состояния после перезапуска.

Сейчас данные вводятся вручную. Настоящая оплата происходит в официальном кабинете Aéza; наша отметка не списывает деньги и не управляет сервером. Интеграция с биллингом - следующий отдельный этап.

Теперь покажем требуемый путь вживую: вход администратора, создание расхода, сообщение Telegram, оплата пользователем, просрочка, исправление даты и следующий независимый период».

Вопросы жюри и короткие ответы

Почему SQLite, а не PostgreSQL

Для текущего масштаба это один сервер и небольшая команда. SQLite даёт транзакции и ограничения уникальности без отдельного сервиса базы, упрощает запуск по README. Мы не называем это решением для распределённого кластера. Для нескольких хостов потребуется PostgreSQL, блокировка строк для заданий и перенос механизма lease. Базу нельзя размещать на сетевом диске.

Зачем API, если уже есть сайт

Сайт использует API при каждом чтении и изменении данных. Оно отделяет экран от серверных правил и даёт проверяемый контракт. Позже к нему можно подключить другой интерфейс или интеграцию. Это API нашего модуля, а не автоматическая связь с биллингом Aéza. API Telegram - ещё один отдельный внешний сервис.

Почему пользователь не сможет повысить себе роль через API

Сервер проверяет сессию и роль на каждом административном маршруте. Модели запросов не разрешают лишние поля для повышения роли. Обычный пользователь получает отказ независимо от того, есть ли кнопка в интерфейсе. Администратор задаётся окружением, через CRUD пользователей он не создаётся и не изменяется.

Что происходит 31-го числа в феврале

Сохраняется исходное число 31. В феврале выбирается последний день - 28-й или 29-й. В марте срок снова 31-го. Мы вычисляем календарные месяцы от исходного числа, а не прибавляем 30 дней и не переносим февральское число на весь год.

Почему единый часовой пояс

Для финансового срока важно единое понятие «сегодня» у всей команды. Деловые даты считаются в `TIMEZONE`, сейчас используется московское время. Технические события и сроки сессий хранятся в UTC. Браузер не должен превращать срок в соседний день из-за своего локального часового пояса.

Как рассчитывается просрочка

Для неоплаченного периода сравниваются текущая дата и срок. Для оплаченного - фактическая дата и срок. Отрицательный результат заменяется нулём. Поэтому оплата в день срока своевременна, а исправление фактической даты может убрать ошибочную просрочку. Срок при этом не меняется.

Что означает состояние unknown

Мы не получили надёжного подтверждения результата отправки. Сообщение могло попасть в Telegram, хотя ответ не дошёл до нас. Слепой повтор способен создать дубль, поэтому автоматического повтора нет. Администратор смотрит диагностику и сверяет чат. Возможен пропуск напоминания, и мы прямо обозначаем этот компромисс.

Можно ли гарантировать доставку ровно один раз

На уровне локальной базы создаётся единственное задание на период, подтверждённые отправки не повторяются. Но у `sendMessage` нет прикладного ключа идемпотентности для нашего задания. При потере ответа абсолютную гарантию ровно одного сообщения дать нельзя. Именно поэтому предусмотрено `unknown`, а не бесконечные повторы.

Что, если оплатить период одновременно с отправкой

Перед отправкой worker снова проверяет отметку и активность серии. Но сообщение, уже переданное внешнему сервису, гарантированно отозвать нельзя. Источником текущего статуса остаётся модуль; напоминание - уведомление о состоянии на момент отправки, а не платёжная операция.

Почему суммы не складываются в один общий итог

По заданию валюты независимы, конвертация не требуется. Без выбранного курса и момента пересчёта общий итог был бы вводящим в заблуждение. Поэтому RUB, USD и EUR показываются отдельно.

Если изменится стоимость услуги, что будет с историей

Администратор может изменить название, сумму и валюту со следующего месяца. Текущий и прошлые периоды, а также уже оплаченные будущие периоды сохраняют прежние снимки. Изменение относится к ручному учёту и не меняет тариф в кабинете провайдера.

Бот может сам подтвердить оплату

Сейчас бот показывает сводки и списки, присылает напоминания и открывает веб-модуль. Подтверждение выполняется в панели с пользовательской сессией. Кнопка «Оплачено» в самом сообщении из дополнительных возможностей задания не реализована.

Что сохраняется после перезапуска

Пользователи, серии, отдельные периоды, даты и статусы оплат, журнал, задания и результаты доставки, а также позиция чтения Telegram. Важно сохранять согласованную копию всей базы вместе с outbox и offset. Для активной SQLite используется backup API или остановка процессов перед копированием.

Что происходит при удалении регулярного платежа

Прекращается его учёт: история остаётся, новые периоды и напоминания не создаются. Это мягкое удаление в нашем модуле. Оно не удаляет VPS и не отключает услугу в Aéza.

Почему это продукт для Aéza, если биллинг ещё не подключён

Реализована адаптация задачи под инфраструктурные расходы, визуальный язык Aéza и связь с её официальным кабинетом через явный переход. Это готовая основа модуля и демонстрация сценария для команды. Полноценное внедрение в платформу требует согласованной интеграции с биллингом, доступа к официальному API и решения о едином входе. Мы не выдаём это за завершённую интеграцию с системами провайдера.

Что нужно, чтобы подключить настоящий биллинг

Получить согласованную документацию и права доступа к API Aéza, определить источники счетов, услуг и подтверждения оплаты, сопоставить внешние идентификаторы с периодами и согласовать авторизацию. Затем реализовать и проверить синхронизацию и обработку повторных событий. Мы не называем конкретные методы чужого API, пока они не подтверждены официальной документацией.

Какие дополнительные функции действительно готовы

Календарь, фильтры и поиск, CSV, журнал действий, диагностика доставки, просмотр в Telegram, изменение будущих условий и адаптивный интерфейс. SMTP, произвольные webhook, вложения чеков, годовая периодичность и оплата через Telegram не реализованы.

Что делал ИИ в проекте

При разработке использовался ИИ: он помогал разбирать требования, писать и перерабатывать код, оформлять интерфейс, готовить проверки и документацию. Это не доказательство корректности само по себе. Результат проверялся автоматическими сценариями, сборкой и живыми проверками стенда. В защите важно объяснить реальные решения и их ограничения, а не приписывать человеку написание каждой строки. Личный вклад выступающего стоит описать только по фактически выполненным им действиям.

Как показать, что решение соответствует заданию

В документации есть таблица соответствия всем страницам кейса, инструкция запуска, спецификация API и результаты проверок. В защите ключевое доказательство - семь обязательных действий вживую. Внешнее оформление и дополнительные функции рассматриваются после обязательной корректности.

Формулировки, которые стоит сохранить точными

- «Напоминание за три календарных дня; поздно добавленный расход - в ближайшую проверку».
- «Единственное задание на период и отсутствие повторов подтверждённой отправки; при неопределённом результате автоматического повтора нет».
- «Отметка об оплате введена пользователем; деньги модуль не списывает».
- «Прекращение учёта сохраняет историю и не отключает услугу у провайдера».
- «Демонстрационные названия и суммы условны; биллинг Aéza не подключён».
- «Поддерживается ежемесячная периодичность; годовые циклы не реализованы».

Источники для сверки: `docs/architecture.md`, `docs/acceptance.md`, `docs/demo.md`, `docs/qa.md`, `README.md`; исходный кейс, страницы 4-17. Перед выступлением нужно синхронизировать номера и названия слайдов с окончательной презентацией и один раз пройти сценарий без чтения подсказок.